

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – CASE STUDY

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO3: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Case Study
Weightage:	20% of CIA	Total Marks:	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	Beffy A Shanika	Reg. No.:	192511387
Section / Batch:	2025	Date:	06/08/2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given case study questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues, provide an in-depth analysis, and propose innovative, feasible solutions with strong justification.

Question Type	Focus Area	Questions
Case Study	Focus on Design Divide and Conquer algorithms: Merge Sort, Quick Sort, Binary Search and Matrix Multiplication	Q1

SECTION A – CASE STUDY

Q1 Financial Data Processing System (Merge vs Quick Sort)

100 marks

A financial system processes and sorts transaction data for reporting and analysis. Analyze the use of Merge Sort and Quick Sort. Identify issues such as stability, speed, and memory usage. Apply divide and conquer principles to compare both algorithms. Analyze performance and justify the most suitable solution.

Code of Conduct

I Beffy A Shanika(192511387) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: beffy

RUBRICS FOR CALCULATION

Case Study					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25%	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20%	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20%	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10%	Clear, well structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%				
Application of Concepts	25%				
Analysis	20%				
Solution Design	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

ANSWER:

CODE:

```
1 def merge_sort(arr):
2     if len(arr) > 1:
3         mid = len(arr) // 2
4         left = arr[:mid]
5         right = arr[mid:]
6         merge_sort(left)
7         merge_sort(right)
8         i = j = k = 0
9         while i < len(left) and j < len(right):
10            if left[i] <= right[j]:
11                arr[k] = left[i]
12                i += 1
13            else:
14                arr[k] = right[j]
15                j += 1
16            k += 1
17        while i < len(left):
18            arr[k] = left[i]
19            i += 1
20            k += 1
21        while j < len(right):
22            arr[k] = right[j]
23            j += 1
24            k += 1
25 def quick_sort(arr, low, high):
26     if low < high:
27         p = partition(arr, low, high)
28         quick_sort(arr, low, p - 1)
29         quick_sort(arr, p + 1, high)
30 def partition(arr, low, high):
31     pivot = arr[high]
32     i = low - 1
33     for j in range(low, high):
34         if arr[j] <= pivot:
35             i += 1
36             arr[i], arr[j] = arr[j], arr[i]
37     arr[i + 1], arr[high] = arr[high], arr[i + 1]
38     return i + 1
39 data = [4500, 1200, 7800, 3400, 2100, 9500, 6700]
40 merge_data = data.copy()
41 quick_data = data.copy()
42 merge_sort(merge_data)
43 quick_sort(quick_data, 0, len(quick_data) - 1)
44 print("Original Data :", data)
45 print("Merge Sort      :", merge_data)
46 print("Quick Sort       :", quick_data)
```

OUTPUT:

```
Original Data : [4500, 1200, 7800, 3400, 2100, 9500, 6700]
Merge Sort    : [1200, 2100, 3400, 4500, 6700, 7800, 9500]
Quick Sort    : [1200, 2100, 3400, 4500, 6700, 7800, 9500]
```

Divide and Conquer Principle:

- Merge Sort: Divides the array into two halves, recursively sorts them, and merges the sorted halves.
- Quick Sort: Selects a pivot, partitions the array into smaller and larger elements, and recursively sorts each partition.

Comparison:

Feature	Merge Sort	Quick Sort
Stability	Stable	Not Stable
Average Time	$O(n \log n)$	$O(n \log n)$
Worst Time	$O(n \log n)$	$O(n^2)$
Memory Usage	$O(n)$	$O(\log n)$
Speed	Consistent	Faster in practice

Performance Analysis:

- Merge Sort provides predictable performance with $O(n \log n)$ even in the worst case, but requires extra memory.
- Quick Sort is usually faster because it sorts in-place and uses less memory, but its worst-case time complexity is $O(n^2)$ if the pivot is poorly chosen.